Український католицький університет

Statistics and Econometrics

Завдання 3.

Лінійна та нелінійна регресія

Задачі 5 і 6

Виконав: **Владислав Григола**

2026

Зміст

Задача 5: Лінійна регресія в Python **3**
(1) Тип житла та кодування категорій . . . 3
(2) Графічний аналіз . . . 5
(3) Проста лінійна регресія . . . 7
(4) Спільний F-тест для MSSubClass . . . 9
(5) Інтерпретація коефіцієнтів . . . 10
(6) Довірчий інтервал і нормальність . . . 12
(7) Покроковий відбір за BIC . . . 14
(8) Викиди, важіль і вплив . . . 15
(9) Імпутація пропусків . . . 18
(10) Ціна чи логарифм ціни . . . 20
(11) Крос-валідація . . . 22

Задача 6: Нелінійні регресії в Python **24**
(а) Лінійна модель . . . 24
(б) Експоненційна модель . . . 26
(в) Статистичні висновки для NLS . . . 28
(г) Збіжність і перенавчання . . . 30

Спільна підготовка

Дані Ames Housing містять 2930 продажів. SalePrice вимірюється в доларах США, площі у квадратних футах. Рівень значущості 0.05, початкове значення генератора 42. Розрахунки виконано в numpy, pandas, scipy, statsmodels і scikit-learn; імпорти та оформлення графіків наведено лише в notebook.

У множинних регресіях використовуємо 2414 повних спостережень. MSSubClass і текстові ознаки кодуємо фіктивними змінними; базовий рівень найчастіший. Не включаємо ідентифікатори Order та PID, GarageYrBlt (не визначений без гаража), а також TotalBsmtSF і GrLivArea, які є точними сумами інших площ. Категоріальні значення NA зберігаємо як окремі рівні.

Код Python

```python
# Вище: імпорти бібліотек, шляхи до файлів, налаштування графіків.
SEED = 42
ALPHA = 0.05
rng = np.random.default_rng(SEED)
DATA_PATH = INPUT_DIR / "AmesHousing.txt"
# Пропуск — лише порожнє поле; текст "NA" лишається значенням категорії.
df = pd.read_csv(DATA_PATH, sep="\t", keep_default_na=False, na_values=[""])
# Назви як у Kaggle та в умові: "Lot Area" -> "LotArea", "Year Remod/Add" -> "YearRemodAdd".
df.columns = df.columns.str.replace(" ", "").str.replace("/", "")
text_columns = list(df.select_dtypes(exclude="number").columns)
for column in text_columns:
    df[column] = df[column].str.strip()  # у SaleType є зайві пробіли, напр. "WD "
# MSSubClass і текстові колонки — категорії; рядок "NA" в них — окремий рівень.
categorical = ["MSSubClass"] + text_columns
# Order, PID — ідентифікатори; GarageYrBlt не існує для будинків без гаража;
# TotalBsmtSF і GrLivArea — точні суми інших площ (див. пункт 4).
excluded = ["Order", "PID", "SalePrice", "GarageYrBlt", "TotalBsmtSF", "GrLivArea"]
numeric = [column for column in df.columns if column not in categorical + excluded]
analysis = df.dropna(subset=numeric + categorical).copy()
# Далі: розмір df, кількість пропусків і структурних рівнів "NA".
```

Задача 5: Лінійна регресія в Python

(1) Тип житла та кодування категорій

Умова завдання

Перед підгонкою моделі до даних спочатку детальніше розглянемо визначення змінних та вирішимо, чи потребують деякі з них окремої обробки. Розглянемо, наприклад, першу змінну MSSubClass. Вона набуває текстових (категоріальних) значень і очевидно не може бути включена в лінійну модель у вихідному вигляді. Існує два підходи до обробки такої змінної. Один підхід полягає в присвоєнні чисел від 1 до 16 кожному можливому результату (Label Encoding). Інший підхід полягає у створенні нових фіктивних змінних (One-Hot Encoding / Dummy variables), які дорівнюють одиниці для конкретного результату і нулю в іншому випадку. Обговоріть ці два підходи та запропонуйте, який із них є більш доречним у цьому випадку. Мотивуйте своє рішення, беручи до уваги номінальну, порядкову та кардинальну шкали зі статистики. Запустіть просту регресію SalePrice на MSSubClass у тому вигляді, в якому вона є (без кодування). Перевірте, як statsmodels або scikit-learn обробляє такий тип змінної, якщо вона передана як звичайний числовий тип (int64), та як її примусово змінити на категоріальну за допомогою pd.Categorical або формул statsmodels.formula.api.ols (C(MSSubClass)).

Код Python

```python
# Вище: print типу MSSubClass (вивід нижче).
numeric_class = smf.ols("SalePrice ~ MSSubClass", data=df).fit()          # як int64
formula_class = smf.ols("SalePrice ~ C(MSSubClass)", data=df).fit()       # C(...) у формулі
df_category = df.assign(MSSubClass=pd.Categorical(df["MSSubClass"]))
category_class = smf.ols("SalePrice ~ MSSubClass", data=df_category).fit()  # pd.Categorical
# Далі: assert рівності прогнозів, збирання таблиць (виводи нижче), графік.
```

```
Тип MSSubClass у DataFrame: int64
```

	Параметрів	R^2	Скоригований R^2
int64: один нахил	2	0.00724058	0.00690152
C(MSSubClass)	16	0.247767	0.243895
pd.Categorical	16	0.247767	0.243895

```
int64-модель: {'Intercept': 189945.2, 'MSSubClass': -159.43}
Перші коефіцієнти C(MSSubClass): {'Intercept': 187356.0, 'C(MSSubClass)[T.30]': -90629.0, 'C(MSSubClass)[T.40]':
-42439.0, 'C(MSSubClass)[T.45]': -75572.0}
```

MSSubClass	count	mean
20	1079	187356
30	139	96726.9
40	6	144917
45	18	111783
50	287	137433
60	575	239364
70	128	156526
75	23	199978
80	118	168009
85	48	149842
90	109	139809
120	192	208019
150	1	148400
160	129	137080
180	17	107671
190	61	125870

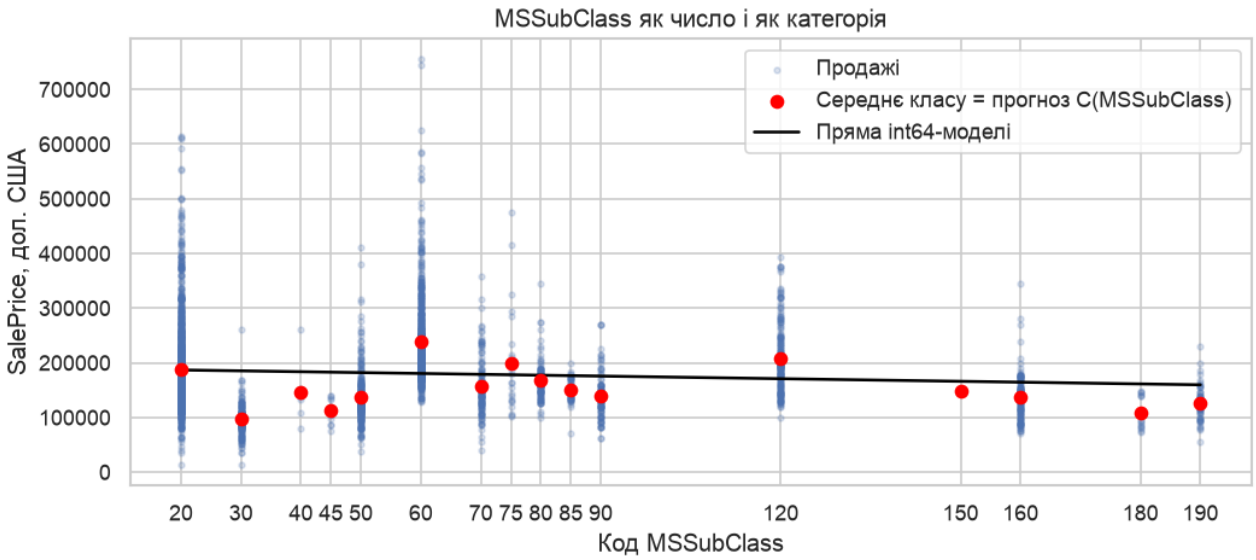


Рис. 1: MSSubClass як числовий код і як категоріальна змінна.

Результати та інтерпретація

`MSSubClass` записано числами (`int64`), але це коди типів житла: код 20 означає одноповерховий будинок 1946 р. і новіший, код 30 такий самий будинок 1945 р. і старший, код 60 двоповерховий будинок, код 90 дуплекс. Це номінальна шкала:

- немає природного порядку (на відміну від порядкової шкали, наприклад оцінки якості 1–10);
- різниця кодів не має змісту (на відміну від кардинальної, кількісної шкали, наприклад площі): $60 - 20 = 40$ нічого не означає.

Label Encoding (номери 1–16 або самі коди) змусив би модель вважати, що кожен крок коду змінює ціну на ту саму суму. One-Hot / фіктивні змінні дають кожному класу власний зсув відносно базового класу. Тому тут доречні фіктивні змінні. Для 16 класів беремо 15 індикаторів і константу: усі 16 разом із константою дали б точну лінійну залежність (пастка фіктивних змінних).

Що робить statsmodels:

- як `int64` змінна вважається звичайним числом, тому оцінюється один нахил: -159.43 дол. на одиницю коду, $R^2 = 0.0072$;
- `C(MSSubClass)` у формулі або `pd.Categorical` перетворюють змінну на 15 фіктивних (базовий клас 20): 16 параметрів, $R^2 = 0.2478$.

Обидва категоріальні способи дають однакові прогнози, а саме середню ціну кожного класу (червоні точки на графіку). Наприклад, коефіцієнт `C(MSSubClass)[T.30]` $= -90629$: клас 30 у середньому дешевший за клас 20 на $187356 - 96727 \approx 90.6$ тис. дол. Чорна пряма int64-моделі майже горизонтальна й не відображає цих різниць.

(2) Графічний аналіз

Умова завдання

Розгляньте такі числові пояснювальні змінні: LotFrontage, LotArea, YearBuilt, BsmtFinSF1, BsmtUnfSF. Побудуйте діаграму розсіювання (scatter plot) залежності SalePrice від цих змінних за допомогою matplotlib або seaborn. Проаналізуйте кожен графік та обговоріть потенційні проблеми, трансформації або розширення моделі, які дозволять врахувати виявлені закономірності. Пам'ятайте, що ви можете або трансформувати наявні змінні (наприклад, np.log), або створювати нові для фіксації нестандартних патернів.

Код Python

```python
scatter_columns = ["LotFrontage", "LotArea", "YearBuilt", "BsmtFinSF1", "BsmtUnfSF"]
units = {"LotFrontage": "фути", "LotArea": "кв. фути", "YearBuilt": "рік",
         "BsmtFinSF1": "кв. фути", "BsmtUnfSF": "кв. фути"}
# П'ять діаграм розсіювання; пропуски (LotFrontage) прибираємо попарно.
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
for ax, column in zip(axes.flat, scatter_columns):
    pairs = df[[column, "SalePrice"]].dropna()
    ax.scatter(pairs[column], pairs["SalePrice"], s=8, alpha=0.3)
    ax.set(title=f"SalePrice та {column} (n={len(pairs)})",
           xlabel=f"{column}, {units[column]}", ylabel="SalePrice, дол. США")
axes.flat[-1].axis("off")
plt.show()
# Далі: збереження рисунка; describe(), нулі, кореляції, групи років (виводи нижче).
```

	count	min	50%	max
LotFrontage	2440	21	68	313
LotArea	2930	1300	9436.5	215245
YearBuilt	2930	1872	1973	2010
BsmtFinSF1	2929	0	370	5644
BsmtUnfSF	2929	0	466	2336

```
Частка нулів: {'BsmtFinSF1': 0.317, 'BsmtUnfSF': 0.083}
Кореляція з SalePrice: {'LotFrontage': 0.357, 'LotArea': 0.267, 'YearBuilt': 0.558, 'BsmtFinSF1': 0.433, 'BsmtUnfSF':
0.183, 'SalePrice': 1.0}
```

Order	Значення	SalePrice	Змінна
1266	313	242000	LotFrontage
1499	313	160000	LotFrontage
957	215245	375000	LotArea
1571	164660	228950	LotArea
1499	5644	160000	BsmtFinSF1
2181	4010	183850	BsmtFinSF1

YearBuilt	(1870, 1920]	(1920, 1950]	(1950, 1980]	(1980, 2000]	(2000, 2010]
count	233	437	1050	475	735
median	119000	126000	142776	200000	226000
std	48793.2	49699.8	41030.8	73249.3	88353.9

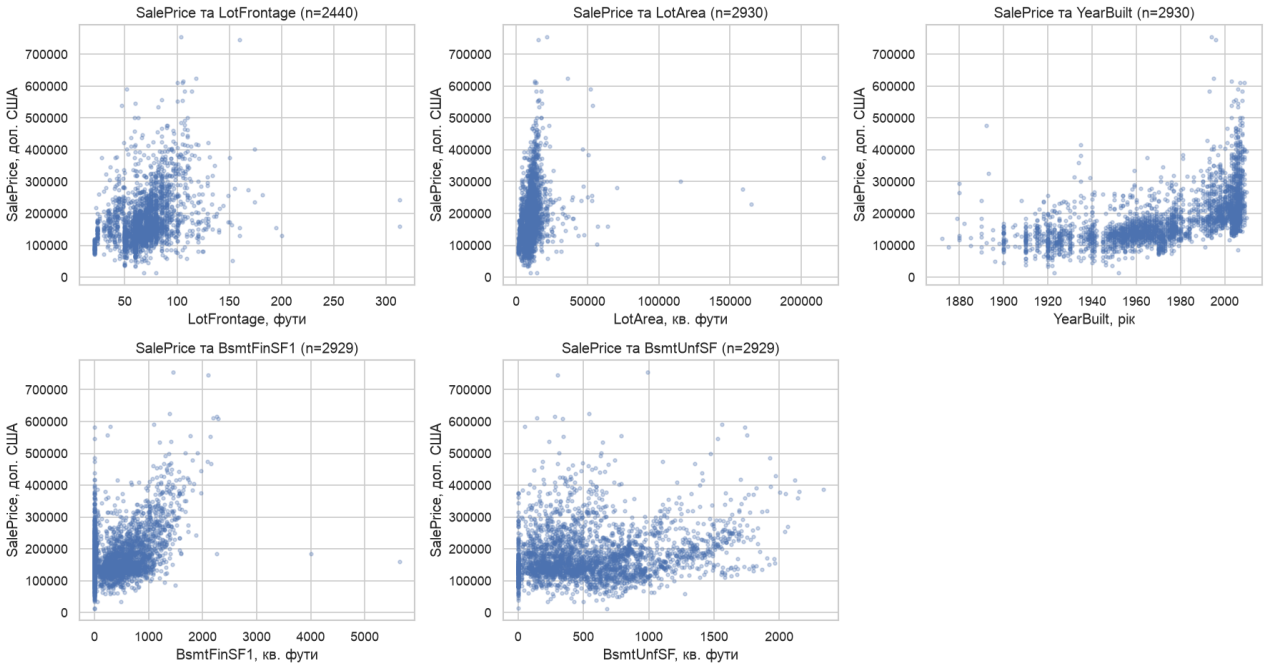


Рис. 2: Ціна продажу та п’ять числових регресорів.

Результати та інтерпретація

- LotFrontage (кореляція 0.357): ціна росте приблизно до 100 футів фасаду, далі зростання слабне; є кілька дуже широких ділянок (до 313 футів). Можна спробувати `np.log(LotFrontage)` або квадратичний член. Також ця змінна має 490 пропусків.
- LotArea (0.267): більшість ділянок менші за 20 тис. кв. футів, але є кілька величезних (до 215245). Розподіл сильно скошений праворуч; такі точки матимуть великий важіль. Ціна з площею росте дедалі повільніше, тому доречний `np.log(LotArea)`.
- YearBuilt (0.558, найсильніший зв’язок): залежність опукла. Медіанна ціна: 119 тис. (до 1920 р.), 126 тис. (1921–1950), 143 тис. (1951–1980), 200 тис. (1981–2000), 226 тис. (після 2000). Для нових будинків більший і розкид (стандартне відхилення 73–88 тис. проти 41–50 тис.). Можна додати $YearBuilt^2$, вік будинку `YrSold - YearBuilt` або моделювати `log(SalePrice)`.
- BsmtFinSF1 (0.433): 31.7% нулів (немає обробленої площі підвалу), тому на графіку є вертикальна смуга біля 0. Для додатних значень зв’язок близький до лінійного. Дві точки з площею 4010 і 5644 мають низьку ціну (Order 2181 і 1499); це потенційні викиди. Розширення: індикатор `BsmtFinSF1 > 0`.
- BsmtUnfSF (0.183): слабкий зв’язок і великий розкид; 8.3% нулів. Окремо ця площа мало пояснює; її краще розглядати разом з іншими площами підвалу.

На всіх графіках розкид цін зростає для дорожчих будинків (гетероскедастичність), а сама ціна скошена праворуч. Це аргументи за `np.log(SalePrice)`. Будинок Order 1499 крайній одразу за кількома змінними. Далі за умовою спочатку оцінюємо просту модель без трансформацій; специфікацію з логарифмом ціни розглянемо окремо.

(3) Проста лінійна регресія

Умова завдання

Підгоніть просту лінійну регресію за допомогою класу OLS з бібліотеки statsmodels без виконання обговорених вище трансформацій. Виберіть один із числових регресорів, наприклад, LotArea. Запишіть відповідну гіпотезу для t-тесту. Наведіть формулу для тест-статистики, поясніть компоненти цієї формули та вкажіть значення для цих компонентів із регресійного звіту (summary()). Оцініть якість підгонки (goodness-of-fit) моделі. Поясніть своїми словами різницю між R^2 та скоригованим (adjusted) R^2.

Код Python

```python
# Проста регресія з константою; summary() дає оцінки, SE, t, p та R².
simple_model = sm.OLS(df["SalePrice"], sm.add_constant(df[["LotArea"]])).fit()
print(simple_model.summary())
beta_1 = simple_model.params["LotArea"]
se_1 = simple_model.bse["LotArea"]
t_1 = (beta_1 - 0) / se_1
p_1 = 2 * stats.t.sf(abs(t_1), df=simple_model.df_resid)
assert np.isclose(t_1, simple_model.tvalues["LotArea"])
assert np.isclose(p_1, simple_model.pvalues["LotArea"])
display(pd.Series({"n": simple_model.nobs, "beta_1": beta_1, "SE(beta_1)": se_1, "t": t_1,
                   "df = n - 2": simple_model.df_resid, "p": p_1,
                   "t_0.975,df": stats.t.ppf(1 - ALPHA / 2, simple_model.df_resid),
                   "R²": simple_model.rsquared, "Скоригований R²": simple_model.rsquared_adj,
                   "Стандартна похибка регресії, дол.":
                        np.sqrt(simple_model.mse_resid)}).to_frame("Значення"))
```

```
OLS Regression Results
==============================================================================
Dep. Variable:              SalePrice   R-squared:                       0.071
Model:                            OLS   Adj. R-squared:                  0.071
Method:                 Least Squares   F-statistic:                     223.9
Date:                Mon, 02 Nov 2026   Prob (F-statistic):           7.63e-49
Time:                        19:59:53   Log-Likelihood:                -37124.
No. Observations:                2930   AIC:                         7.425e+04
Df Residuals:                    2928   BIC:                         7.426e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       1.534e+05   2319.909     66.112      0.000    1.49e+05    1.58e+05
LotArea        2.7022      0.181     14.965      0.000       2.348       3.056
==============================================================================
Omnibus:                      981.022   Durbin-Watson:                   0.771
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             4394.498
Skew:                           1.561   Prob(JB):                         0.00
Kurtosis:                       8.124   Cond. No.                     2.09e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.09e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

	Значення
n	2930
beta_1	2.70224
SE(beta_1)	0.180575
t	14.9646
df = n - 2	2928
p	7.63384e-49
t_0.975,df	1.96077
R^2	0.0710485
Скоригований R^2	0.0707312
Стандартна похибка регресії, дол.	77009.6

Результати та інтерпретація

Модель: $SalePrice_i = \beta_0 + \beta_1 LotArea_i + \varepsilon_i$, де β_0 позначає вільний член, β_1 нахил, ε_i похибку продажу i; $n = 2930$.

Гіпотези t-тесту: $H_0 : \beta_1 = 0$ (площа ділянки лінійно не пов'язана з ціною) проти $H_1 : \beta_1 \neq 0$.

$$t = \frac{\widehat{\beta}_1 - 0}{SE(\widehat{\beta}_1)}, \qquad SE(\widehat{\beta}_1) = \sqrt{\frac{\widehat{\sigma}^2}{\sum_i (x_i - \bar{x})^2}}, \qquad df = n - 2.$$

де $\widehat{\beta}_1$ позначає оцінку нахилу, $SE(\widehat{\beta}_1)$ її стандартну похибку, $\widehat{\sigma}^2 = RSS/(n-2)$ оцінку дисперсії похибок, x_i значення `LotArea`, $\bar{x}$ їх середнє, n кількість спостережень, df кількість ступенів вільності.

Компоненти з `summary()`:

- $\widehat{\beta}_1 = 2.7022$ (`coef`), оцінений нахил: +1 кв. фут ділянки відповідає в середньому +2.70 дол. ціни;
- $SE(\widehat{\beta}_1) = 0.1806$ (`std err`), тобто наскільки оцінка коливалася б від вибірки до вибірки;
- 0, значення β_1 за H_0;
- $t = 2.7022/0.1806 = 14.96$ (`t`), $df = 2928$ (`Df Residuals`);
- $p = 7.6 \cdot 10^{-49}$ (`P>|t|` показує 0.000).

$|t| = 14.96 > t_{0.975,2928} = 1.961$, тому H_0 відхиляємо: площа ділянки статистично значуща.

Якість підгонки слабка: $R^2 = 0.071$, тобто площа ділянки пояснює лише 7.1% варіації цін. Стандартна похибка регресії $\widehat{\sigma} = \sqrt{RSS/(n-2)}$ становить близько 77 тис. дол.: це типовий розмір залишку. F-статистика 223.9 для однієї змінної дорівнює t^2. Значущий коефіцієнт ще не означає добру модель. Також `summary()` показує ненормальні залишки (Jarque–Bera = 4394). Великий `Cond. No.` тут пов'язаний лише з масштабом: значення `LotArea` сягають десятків тисяч кв. футів, а константа дорівнює 1. З одним регресором мультиколінеарності немає.

R^2 і скоригований R^2:

$$R^2 = 1 - \frac{RSS}{TSS}, \qquad \bar{R}^2 = 1 - \frac{RSS/(n-k)}{TSS/(n-1)}.$$

де RSS позначає суму квадратів залишків, TSS суму квадратів відхилень ціни від її середнього, k кількість параметрів разом із константою, n кількість спостережень.

R^2 показує частку поясненої варіації; він ніколи не зменшується при додаванні змінної, навіть зовсім шумової. Скоригований R^2 штрафує за кількість параметрів k і зростає лише тоді, коли нова змінна достатньо зменшує RSS. Тут $k = 2$, тому різниця мізерна: 0.0710 проти 0.0707. Для повної моделі з 269 параметрами різниця вже помітна: 0.9399 проти 0.9324.

(4) Спільний F-тест для MSSubClass

Умова завдання

Якщо ви хочете стверджувати, що змінна MSSubClass є незначущою, і використовуєте модель із фіктивними змінними, то вам необхідно перевірити одночасну (спільну) незначущість усіх даммі-змінних, що походять від факторної змінної MSSubClass. Запустіть тест для загальної лінійної гіпотези за допомогою методу f_test() у statsmodels та зробіть висновок про значущість MSSubClass. Запишіть матрицю та вектор, які необхідні для формулювання цієї гіпотези в межах лінійної моделі.

Код Python

```python
# Вище: assert, що TotalBsmtSF і GrLivArea є сумами інших площ.
# Зберігаємо повні блоки кожної початкової змінної, навіть якщо індикатори збігаються.
# BldgType перед MSSubClass: спільний індикатор Duplex належить контрольній змінній.
category_order = ["BldgType"] + [c for c in categorical if c != "BldgType"]
groups = {column: [column] for column in numeric}
blocks = [analysis[numeric].astype(float)]
for column in category_order:
    dummies = pd.get_dummies(analysis[column], prefix=column, dtype=float)
    dummies = dummies.drop(columns=f"{column}_{analysis[column].mode()[0]}")
    groups[column] = list(dummies.columns)
    blocks.append(dummies)
X_encoded = pd.concat(blocks, axis=1)
def design_matrix(active):
    """Матриця активних змінних: прибираємо копії лише в цій моделі."""
    columns = [c for group in active for c in groups[group]]
    X = X_encoded[columns]
    return sm.add_constant(X.loc[:, ~X.T.duplicated()], has_constant="add")
X_full = design_matrix(list(groups))
y = analysis["SalePrice"].astype(float)
full_model = sm.OLS(y, X_full).fit()
# Коефіцієнти MSSubClass, які справді оцінюються (без копії індикатора дуплексу).
dummy_names = [name for name in X_full.columns if name.startswith("MSSubClass_")]
q, k = len(dummy_names), X_full.shape[1]
positions = X_full.columns.get_indexer(dummy_names)
# Матриця обмежень: один рядок на кожен коефіцієнт MSSubClass; вектор r = 0.
R = np.zeros((q, k))
R[np.arange(q), positions] = 1
r = np.zeros(q)
# f_test() перевіряє H0: R beta = r у повній моделі.
f_result = full_model.f_test((R, r))
# Далі: assert рангу, модель без MSSubClass, ручний F і assert збігу з f_test().
```

```
Вилучені копії стовпців: ['BsmtCond_NA', 'BsmtExposure_NA', 'BsmtFinType1_NA', 'BsmtFinType2_NA', 'Exterior2nd_PreCast',
'GarageCond_NA', 'GarageFinish_NA', 'GarageQual_NA', 'MSSubClass_90', 'Utilities_NoSewr']
Повна модель: n = 2414, k = 269 параметрів, R² = 0.9399, скоригований R² = 0.9324
R має розмір 13x269; одиниці в стовпцях [37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49] (рахуючи з 0); r = 0 розміру
13
```

	Спільний F-тест
F	1.40493
df1 = q	13
df2 = n - k	2145
p	0.148857
F_0.95	1.72472
RSS без MSSubClass	1.01765e+12
RSS повної моделі	1.00905e+12

Результати та інтерпретація

Повна модель має $n = 2414$, $k = 269$ параметрів: константу, 32 числові регресори та 236 фіктивних змінних. $R^2 = 0.9399$.

Для єдиного розв'язку МНК прибираємо точні лінійні залежності: `TotalBsmtSF` і `GrLivArea` є сумами інших площ, а частина фіктивних змінних збігається. При цьому важливо зберегти зміст змінних, які контролюємо.

Особливість MSSubClass: у робочій вибірці є 15 класів, тобто 14 індикаторів відносно класу 20. Але `MSSubClass_90` точно збігається з `BldgType_Duplex`. Якщо ми перевіряємо внесок MSSubClass понад усі інші змінні, індикатор дуплексу має залишатися в обох моделях. Тому залишаємо його під назвою `BldgType_Duplex`, а для MSSubClass маємо 13 незалежних обмежень. Єдиний продаж класу 150 не входить у робочу вибірку через пропущений `LotFrontage`.

Гіпотеза: H_0: усі 13 окремо оцінюваних коефіцієнтів MSSubClass дорівнюють нулю; H_1: хоча б один не дорівнює нулю. У матричному вигляді:

$$H_0 : R\beta = r, \qquad R = \big[\, 0_{13\times 37} \mid I_{13} \mid 0_{13\times 219} \,\big], \qquad r = 0_{13}.$$

Тут β позначає вектор 269 коефіцієнтів повної моделі, R матрицю обмежень розміру 13×269, I_{13} одиничну матрицю, $0_{13\times 37}$ і $0_{13\times 219}$ нульові блоки, r нульовий вектор довжини 13. У порядку `X_full` перед MSSubClass стоять константа, 32 числові регресори й 4 індикатори BldgType. Одиниці R вибирають стовпці 37–49 (індексація від 0), як показує вивід коду.

`f_test()` перевіряє ці обмеження. Той самий результат отримуємо, порівнюючи моделі з MSSubClass і без нього, але з усіма іншими змінними в обох:

$$F = \frac{(RSS_R - RSS_U)/13}{RSS_U/(2414 - 269)} = 1.405, \qquad p = 0.149.$$

де RSS_R позначає суму квадратів залишків обмеженої моделі без MSSubClass, RSS_U повної моделі, 13 кількість обмежень, $2414 - 269$ кількість ступенів вільності залишків повної моделі.

На рівні 0.05 не відхиляємо H_0: дані не дають достатніх доказів, що MSSubClass додає пояснення ціни після врахування BldgType, HouseStyle, YearBuilt та решти регресорів. Це не доводить відсутності будь-якого зв'язку типу житла з ціною: частину інформації вже містять інші змінні.

Це класичний F-тест за припущення однакової дисперсії та нормальності похибок; якщо вони порушені, p-значення є лише наближеним.

(5) Інтерпретація коефіцієнтів

Умова завдання

Дайте економічну інтерпретацію параметрів для LotArea, першої фіктивної змінної для MSSubClass (це підступне питання!) та BedroomAbvGr. (Ігноруйте потенційну незначущість коефіцієнтів.) Як зміниться інтерпретація, якщо моделювати логарифм від SalePrice (np.log(df['SalePrice'])), використовуючи лог-лінійну специфікацію?

Код Python

```python
# Та сама матриця X_full, але відгук — log(SalePrice).
full_log_model = sm.OLS(np.log(y), X_full).fit()
first_dummy = dummy_names[0]
terms = ["LotArea", first_dummy, "BedroomAbvGr"]
gamma = full_log_model.params[terms]
# Далі: збирання таблиць коефіцієнтів і років побудови класів 20 і 30 (виводи нижче).
```

	beta (SalePrice), дол.	p (SalePrice)	gamma (log SalePrice)	100*gamma, %	100*(exp(gamma)-1), %
LotArea	0.798701	5.71237e-13	3.06586e-06	0.000306586	0.000306587
MSSubClass_30	5508.22	0.0913135	-0.0413429	-4.13429	-4.05
BedroomAbvGr	-3263.73	0.000872302	-0.00100473	-0.100473	-0.100423

```
LotArea +1000 кв. футів: 799 дол.; 0.31% у лог-моделі
Базова категорія MSSubClass: 20 | перша фіктивна: MSSubClass_30
```

MSSubClass	count	min	max
20	881	1938	2010
30	126	1885	1948

Результати та інтерпретація

Усі інтерпретації подано за незмінних інших змінних повної моделі.

Модель для `SalePrice` (у доларах):

- LotArea: $\widehat{\beta} = 0.80$. Додатковий кв. фут ділянки відповідає в середньому +0.80 дол., а +1000 кв. футів дають +799 дол. Це значно менше, ніж 2.70 у простій регресії (пункт 3): там `LotArea` частково відображала розмір будинку, район тощо.
- Перша фіктивна `MSSubClass_30`: $\widehat{\beta} = 5508$. Підступ у тому, що це не ефект «збільшення коду на одиницю» і не ціна класу 30. Це різниця з базовим класом 20: будинок класу 30 (одноповерховий, 1945 р. і старший) у середньому на 5.5 тис. дол. дорожчий за будинок класу 20 (одноповерховий, 1946 р. і новіший) за інших рівних. Інша база дала б інший коефіцієнт. Крім того, класи 20 і 30 визначено через рік побудови, а `YearBuilt` уже є в моделі. «Однаковий рік» для двох класів трапляється рідко: у вибірці клас 30 збудовано у 1885–1948 рр., а клас 20 у 1938–2010 рр. Коефіцієнт оцінено на всій вибірці, але перекриття років між класами обмежене, тому порівняння «за однакового року» сильно залежить від лінійної форми моделі. За звичайним t-тестом він незначущий на рівні 0.05 ($p = 0.09$). Проста різниця середніх у пункті 1 була -90.6 тис., бо вона включала й вік, і розмір будинку.
- BedroomAbvGr: $\widehat{\beta} = -3264$. Ще одна спальня за тих самих площ поверхів відповідає ціні, нижчій у середньому на 3.3 тис. дол.: та сама площа ділиться на більше дрібних кімнат. Без контролю площі зв'язок кількості спалень із ціною був би додатним.

Лог-лінійна модель $\log(SalePrice) = X\gamma + u$, де X позначає ту саму матрицю регресорів, γ вектор коефіцієнтів, u похибку: коефіцієнт γ_j показує зміну логарифма ціни. Тоді ціна змінюється приблизно на $100\gamma_j$%, а точно на $100(e^{\gamma_j} - 1)$%:

- LotArea: $\widehat{\gamma} = 3.07 \cdot 10^{-6}$, тобто +1000 кв. футів дають приблизно +0.31% ціни;
- MSSubClass_30: $\widehat{\gamma} = -0.0413$, тобто клас 30 приблизно на 4.1% (точно на 4.05%) дешевший за клас 20. Знак відрізняється від моделі ціни в доларах: ці дві специфікації по-різному описують залежність ціни від ознак.
- BedroomAbvGr: $\widehat{\gamma} = -0.0010$, тобто приблизно -0.10% на спальню.

У лог-моделі ефекти відсоткові, а не в доларах: та сама зміна регресора дає більше доларів для дорожчого будинку. Для фіктивних змінних із великим γ варто брати точну формулу $100(e^{\gamma} - 1)$%. Ці відсотки описують зміну прогнозу $e^{X\gamma}$, тобто умовної медіани ціни за симетричних похибок. Для середньої ціни потрібне ще припущення, що множник $E[e^u \mid X]$ не залежить від X.

(6) Довірчий інтервал і нормальність

Умова завдання

Обчисліть 95% довірчий інтервал для параметра LotArea (використовуючи метод conf_int()) та поясніть його економічний зміст. Зв'яжіть довірчі інтервали з тестами значущості, тобто поясніть, як би ви використовували ці інтервали для прийняття рішення про значущість відповідних пояснювальних змінних? Довірчі інтервали обчислюються на основі припущення, що залишки (residuals) підпорядковуються нормальному розподілу. Чи виконується це припущення? Запустіть відповідний критерій згоди за допомогою scipy.stats.shapiro або jarque_bera.

Код Python

```python
# conf_int() будує інтервал beta ± t_{0.975, n-k} * SE.
ci = full_model.conf_int(alpha=ALPHA).loc["LotArea"]
# H0 Jarque-Bera: асиметрія 0 та ексцес 3, як у нормального розподілу.
jb = stats.jarque_bera(full_model.resid)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(full_model.resid, bins=60, edgecolor="white")
sm.qqplot(full_model.resid, line="s", ax=axes[1], markersize=3)
plt.show()
# Далі: ручний перерахунок інтервалу з assert, підписи осей, збереження рисунка.
```

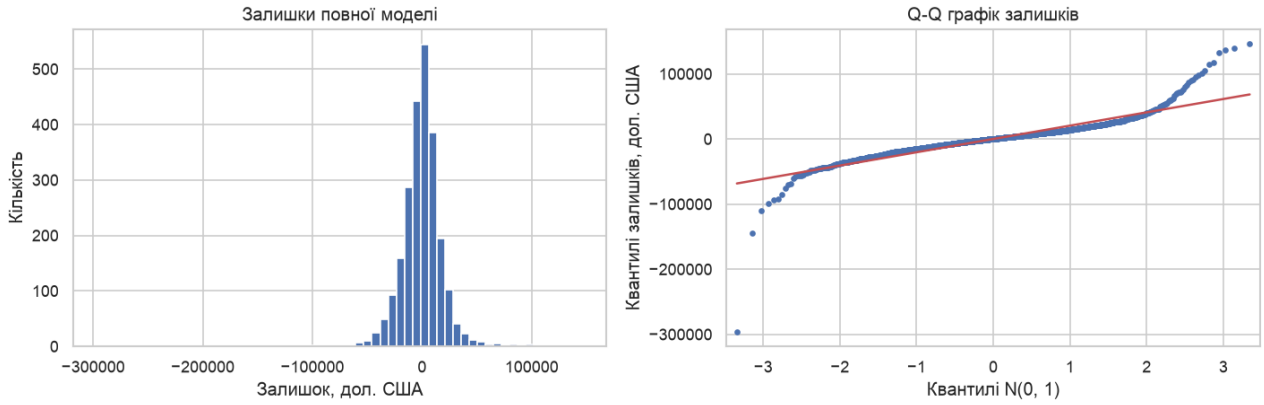


Рис. 3: Розподіл і Q-Q графік залишків повної моделі.

	LotArea, повна модель
beta_LotArea	0.798701
SE	0.110137
t_0.975,n-k	1.96107
Нижня межа	0.582715
Верхня межа	1.01469
p t-тесту	5.71237e-13

	Jarque-Bera для залишків
JB	66037.7
p	0
Асиметрія	-0.595274
Ексцес (нормальний = 3)	28.5955

Результати та інтерпретація

95% довірчий інтервал для $\beta_{LotArea}$ у повній моделі:

$$\widehat{\beta} \pm t_{0.975,\,n-k}\, SE(\widehat{\beta}) = 0.7987 \pm 1.961 \cdot 0.1101 = [0.583;\ 1.015] \text{ дол. за кв. фут.}$$

де $\widehat{\beta}$ позначає оцінку коефіцієнта, $SE(\widehat{\beta})$ її стандартну похибку, $t_{0.975,\,n-k}$ квантиль t-розподілу з $n-k=2145$ ступенями вільності.

Економічний зміст: за інших рівних додаткові 1000 кв. футів ділянки пов'язані зі зростанням середньої ціни приблизно на 583–1015 дол. Число «95%» стосується процедури: якби ми багато разів брали нові вибірки й будували інтервал так само, близько 95% інтервалів накрили б справжнє β (якщо припущення моделі виконуються). Це інтервал для коефіцієнта, а не для ціни окремого будинку.

Зв'язок із тестом: двосторонній t-тест $H_0 : \beta = \beta_0$ на рівні α відхиляє H_0 тоді й лише тоді, коли β_0 не потрапляє в $(1-\alpha)$-інтервал. Для значущості беремо $\beta_0 = 0$. Тут 0 лежить поза $[0.583;\ 1.015]$, тому `LotArea` значуща на рівні 0.05 ($p = 5.7 \cdot 10^{-13}$). Інтервал дає більше, ніж p-значення: він показує ще й правдоподібний розмір ефекту.

Нормальність залишків. Тест Jarque–Bera: $JB = \frac{n}{6}\big(S^2 + \frac{(K-3)^2}{4}\big)$, де n позначає кількість залишків, S їхню асиметрію, K ексцес (куртозис); H_0: розподіл нормальний. Отримали $JB = 66038$, $p \approx 0$ (менше за точність обчислень). Асиметрія -0.60, ексцес 28.6 (у нормального 3): хвости дуже важкі. На Q–Q графіку крайні точки далеко від прямої, окремі залишки сягають -300 тис. дол. Припущення нормальності не виконується.

Тому точні t-інтервали тут лише наближення. Для $n = 2414$ вони залишаються наближено правильними, якщо інші припущення (незалежність, однаковий розкид похибок) виконуються. Проте гістограма та Q–Q графік показують важкі хвости й кілька дуже великих залишків. Тому стандартні похибки й інтервали слід сприймати обережно.

(7) Покроковий відбір за BIC

Умова завдання

Більшість змінних можуть виявитися незначущими, тому нам слід знайти найменшу (найбільш економну) модель, яка все ще має хорошу пояснювальну здатність. Виберіть таку модель за допомогою покрокового відбору змінних (stepwise model selection), реалізувавши власний алгоритм на основі метрик AIC/BIC або скориставшись методами регуляризації Lasso з sklearn.linear_model. Детально поясніть, як цей метод/підхід працює у вашому коді. Працюйте з цією обраною моделлю на всіх наступних кроках.

Код Python

```python
def backward_bic(y, groups):
    """Зворотний відбір початкових змінних за BIC."""
    active = list(groups)
    model = sm.OLS(y, design_matrix(active)).fit()
    history = [{"Крок": 0, "Вилучено": "–", "k": int(model.df_model + 1), "BIC": model.bic}]
    while active:
        best_group, best_model = None, None
        # Пробуємо вилучити кожну змінну окремо; запам'ятовуємо кандидата з найменшим BIC.
        for group in active:
            remaining = [g for g in active if g != group]
            candidate = sm.OLS(y, design_matrix(remaining)).fit()
            if best_model is None or candidate.bic < best_model.bic:
                best_group, best_model = group, candidate
        # Зупиняємося, коли жодне вилучення не зменшує BIC.
        if best_model.bic >= model.bic:
            break
        active.remove(best_group)
        model = best_model
        history.append({"Крок": len(history), "Вилучено": best_group,
                        "k": int(model.df_model + 1), "BIC": model.bic})
    return model, active, pd.DataFrame(history)
# Починаємо з усіх початкових змінних; категорію вилучаємо цілим блоком індикаторів.
selected_model, selected_terms, history = backward_bic(y, groups)
X_selected = design_matrix(selected_terms)
# Далі: assert рангу і локального оптимуму BIC; друк історії та списку змінних.
```

Крок	Вилучено	k	BIC
0	-	269	56866.2
1	Exterior2nd	254	56759.9
2	MSSubClass	241	56679.4
3	Exterior1st	226	56606.7
4	SaleType	217	56546
…	…	…	…
44	GarageFinish	95	55905.8
45	Street	94	55902.6
46	HalfBath	93	55900.6
47	KitchenAbvGr	92	55898.9
48	YearRemodAdd	91	55897.5

Скорочено: з 49 рядків історії показано кроки 0–4 і 44–48; рядок «…» позначає пропущені кроки 5–43.

```
Залишилось 28 з 76 змінних: ['LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'MasVnrArea', 'BsmtFinSF1',
'BsmtFinSF2', 'BsmtUnfSF', '1stFlrSF', '2ndFlrSF', 'FullBath', 'BedroomAbvGr', 'Fireplaces', 'GarageArea',
'ScreenPorch', 'BldgType', 'LandSlope', 'Neighborhood', 'Condition2', 'RoofMatl', 'MasVnrType', 'ExterQual', 'BsmtQual',
'BsmtExposure', 'KitchenQual', 'PoolQC', 'MiscFeature', 'SaleCondition']
```

	Змінних	Параметрів k	R^2	Скоригований R^2	AIC	BIC
Повна	76	269	0.939889	0.932378	55308.9	56866.2
Після відбору	28	91	0.928533	0.925764	55370.7	55897.5

Результати та інтерпретація

Використано власний зворотний покроковий відбір за BIC:

$$BIC = -2\ln\widehat{L} + k\ln n, \qquad -2\ln\widehat{L} = n\ln(RSS/n) + \text{const}.$$

Тут $\widehat{L}$ позначає максимум функції правдоподібності, k кількість незалежних параметрів разом із константою, n кількість спостережень, RSS суму квадратів залишків. Для $n = 2414$ штраф за параметр $\ln n \approx 7.8$, а в AIC лише 2. BIC сильніше штрафує складність.

Як працює код:

1. Починаємо з усіх 76 початкових регресорів: 32 числових і 44 категоріальних.
2. Для кожного можливого вилучення заново будуємо матрицю з повних блоків решти початкових змінних та прибираємо дубльовані стовпці лише в цій кандидатній моделі.
3. Оцінюємо моделі та вилучаємо змінну, без якої BIC найменший, якщо це покращує поточний BIC.
4. Повторюємо до зупинки.

Категоріальну змінну розглядаємо цілком. Якщо два фактори мають спільний індикатор, він залишається, доки потрібен хоча б одному з них. Наприклад, після вилучення MSSubClass у моделі лишається `BldgType_Duplex`. Так само відновлюються потрібні індикатори відсутності підвалу чи гаража. Тому відбір стосується початкових змінних, а не довільного розподілу спільних стовпців між ними. Усі моделі використовують ті самі 2414 рядків.

Результат: за 48 кроків вилучено 48 регресорів, залишилося 28 змінних і 91 параметр. Серед них площі, рік будівництва, кількість спалень і ванних кімнат, якість будинку, тип будівлі та район. Повний список і скорочену історію кроків наведено у виводі коду.

Обрана модель має майже втричі менше параметрів, втрачає приблизно 0.011 у R^2, а BIC зменшується приблизно на 969. AIC із цих двох моделей віддає перевагу повній, бо слабше штрафує складність. Покроковий відбір перевіряє не всі комбінації, тому не гарантує глобально найменшого BIC.

У вибраній моделі лишилися рідкісні категорії (`RoofMatl`, `PoolQC`, `MiscFeature` тощо). Їхню роль варто перевірити через важіль і відстань Кука, а якість прогнозів через крос-валідацію.

(8) Викиди, важіль і вплив

Умова завдання

Іноді дані містять викиди (outliers), що спричиняє зсув в оцінках параметрів. Перевірте наявність викидів за допомогою графіків залишків, відстані Кука (Cook's distance) та важеля (leverage). Для обчислення цих показників скористайтеся класом OLSInfluence з бібліотеки statsmodels. Зверніть прискіпливу увагу на спостереження з найбільшим значенням важеля (leverage). Що саме робить це спостереження таким видатним (ви можете побудувати sns.boxplot для інтервальних змінних або порахувати частоти через value_counts())?

Код Python

```python
influence = OLSInfluence(selected_model)
with np.errstate(invalid="ignore", divide="ignore"):  # для h = 1 формули містять 0/0
    leverage = pd.Series(influence.hat_matrix_diag, index=y.index)
    student = pd.Series(influence.resid_studentized_internal, index=y.index)
    cooks = pd.Series(influence.cooks_distance[0], index=y.index)
# h = 1: модель точно підганяє продаж, його залишок дорівнює нулю.
exact_fit = leverage > 1 - 1e-8
cooks[exact_fit] = np.nan    # h = 1: залишок 0, D = 0/0 не визначено
student[exact_fit] = np.nan
# Чутливість: оцінки без спостереження з найбільшим Cook's D.
top_cook = cooks.idxmax()
without_top = sm.OLS(y.drop(top_cook), X_selected.drop(top_cook)).fit()
# Далі: assert суми важелів, пошук рідкісних рівнів для h = 1, таблиці, графіки.
```

```
k = 91, n = 2414, середній важіль k/n = 0.0377, орієнтир 2k/n = 0.0754
Спостережень з h > 2k/n: 163 | з h = 1: 8
```

Order	SalePrice	h	Залишок	Рідкісні рівні (кількість продажів)
211	107500	1	3.31056e-08	KitchenQual=Po (1)
1402	241500	1	5.76838e-08	RoofMatl=Membran (1)
1499	160000	1	9.62755e-08	RoofMatl=ClyTile (1)
1908	137000	1	3.45754e-08	RoofMatl=Roll (1)
2181	183850	1	5.73054e-08	MiscFeature=Elev (1)
2299	80000	1	5.13101e-08	MasVnrType=CBlock (1)
2351	250000	1	1.55123e-08	PoolQC=Fa (2), MiscFeature=TenC (1)
2358	181000	1	2.90456e-08	PoolQC=Fa (2)

```
RoofMatl.value_counts(): {'CompShg': 2384, 'Tar&Grv': 15, 'WdShngl': 7, 'WdShake': 5, 'Membran': 1, 'ClyTile': 1,
'Roll': 1}
```

Order	Neighborhood	SaleCondition	1stFlrSF	SalePrice	Прогноз	Залишок	h	Cook_D
2182	Edwards	Partial	3138	184750	489336	-304586	0.294699	1.16948
514	Somerst	Normal	2490	468000	324780	143220	0.269261	0.220088
1761	NoRidge	Abnorml	2411	745000	625485	119515	0.291387	0.176375
2571	NAmes	Abnorml	1360	235000	344412	-109412	0.279098	0.136797
1015	StoneBr	Normal	1710	334000	239299	94701	0.281954	0.104358

	Order	Neighbor-hood	Sale Condition	LotArea	1stFlrSF	2ndFlrSF	YearBuilt	SalePrice	h	Cook_D
2180	2181	Edwards	Partial	39290	5095	0	2008	183850	1	NaN
1498	1499	Edwards	Partial	63887	4692	950	2008	160000	1	NaN
2181	2182	Edwards	Partial	40094	3138	1538	2007	184750	0.294699	1.16948
956	957	Timber	Normal	215245	2036	0	1965	375000	0.662552	0.0219219
Медіана вибірки	NaN	NaN	NaN	9245.5	1073	0	1972	157000	NaN	NaN

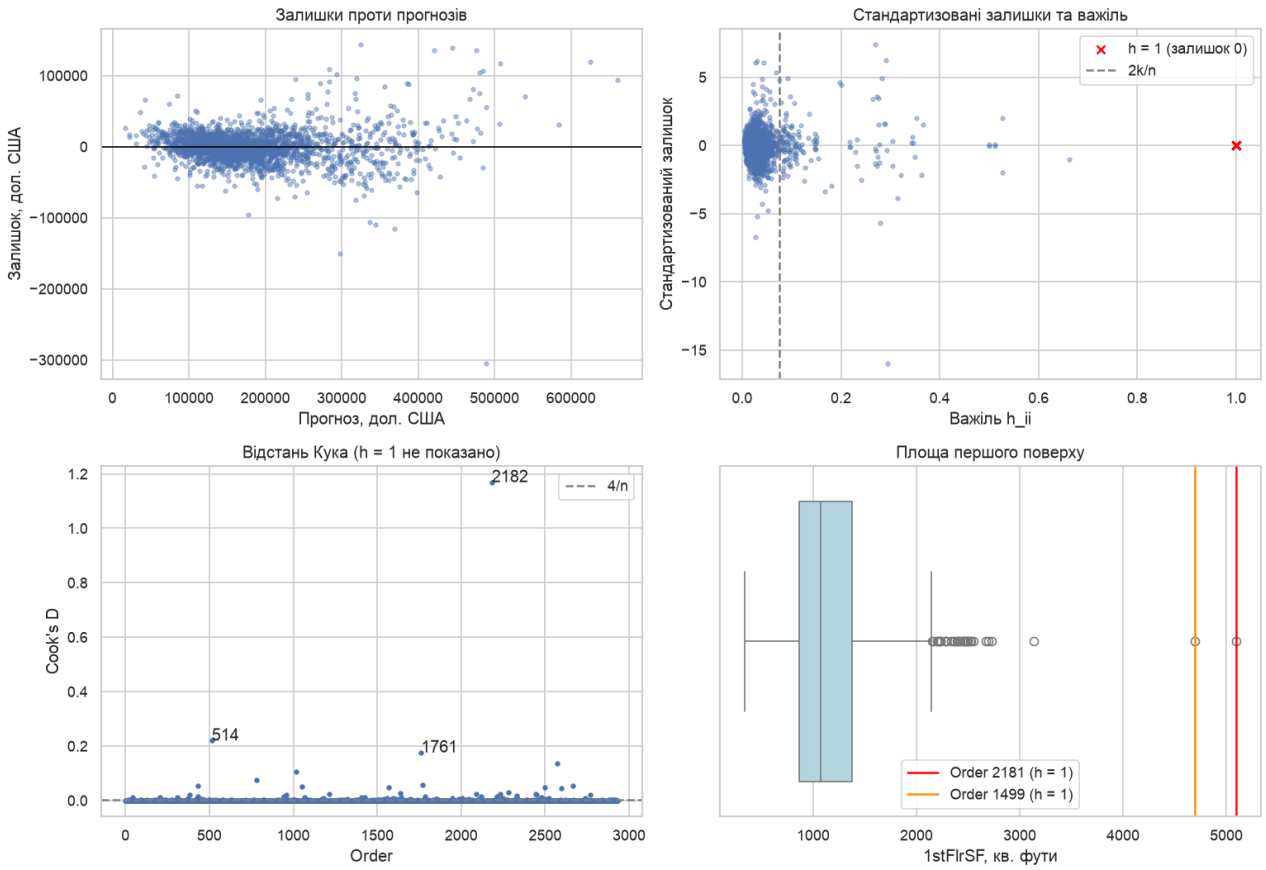


Рис. 4: Залишки, важіль, відстань Кука та нетипові площі.

	Усі продажі	Без Order 2182
LotArea	0.803024	0.932567
1stFlrSF	48.6615	51.9886
2ndFlrSF	55.525	60.1981
BsmtFinSF1	42.6449	44.7811
OverallQual	6978.72	6976.48
BedroomAbvGr	-4143.96	-5228.88

Результати та інтерпретація

Важіль h_{ii} є діагоналлю матриці $H = X(X^TX)^{-1}X^T$, де X позначає матрицю регресорів відібраної моделі. Він залежить лише від регресорів і показує, наскільки незвичний набір ознак будинку. Маємо $0 \le h_{ii} \le 1$, $\sum_i h_{ii} = k = 91$. Середній важіль $k/n = 0.0377$, орієнтир для великого важеля $2k/n = 0.0754$ перевищують 163 продажі.

Відстань Кука поєднує залишок і важіль:

$$D_i = \frac{e_i^2}{k\,s^2}\,\frac{h_{ii}}{(1-h_{ii})^2}.$$

де e_i позначає залишок спостереження i, $s^2 = RSS/(n-k)$ оцінку дисперсії похибок, k кількість параметрів. Вона вимірює зміну підгонки після вилучення спостереження. Значення $4/n$ і 1 є діагностичними орієнтирами, а не автоматичними правилами видалення даних.

Найбільший важіль $h = 1$ мають 8 продажів. Причина в рідкісних категоріях. Наприклад, Order 1499 є єдиним продажем із `RoofMatl=ClyTile`, а Order 2181 із `MiscFeature=Elev`. Order 2358 має `PoolQC=Fa`: таких продажів два, але інший уже виділено унікальним індикатором `TenC`. Комбінація цих індикаторів дозволяє точно підлаштувати обидва продажі. Залишки в усіх восьми точках практично нульові; для $h = 1$ стандартна формула Кука не визначена, тому показуємо `NaN`, а не помилковий нуль.

Особливо незвичні Order 1499 і 2181: будинки 2008 року в Edwards із площами першого поверху 4692 та 5095 кв. футів за медіани 1073. Їхні ціни становлять лише 160000 і 183850 дол. Обидва мають `SaleCondition=Partial`: будинок не був завершений під час останньої оцінки, згідно з описом даних. Рідкісні індикатори даху ClyTile та ліфта Elev дозволяють моделі точно підігнати ці продажі.

Серед $h < 1$ найбільший важіль має Order 957, $h = 0.663$: ділянка 215245 кв. футів, приблизно у 23 рази більша за медіану. Проте його відстань Кука $D = 0.022$ значно менша, ніж у найвпливовіших точок. Великий важіль сам по собі не означає великої відстані Кука.

Найбільша визначена відстань Кука в Order 2182: $D = 1.17$, $h = 0.295$. Фактична ціна 184750 дол., прогноз 489336 дол., залишок приблизно -305 тис. Без цього продажу коефіцієнт LotArea змінюється з 0.803 на 0.933, BedroomAbvGr з -4144 на -5229. Це показує чутливість оцінок.

На графіку залишків розкид загалом більший для високих прогнозів, що вказує на можливу гетероскедастичність. У даних є незвичні й впливові продажі; автоматично їх не вилучаємо. Варто враховувати чутливість висновків, а для покращення прогнозів дослідити великі будинки `Partial` та об'єднання рідкісних рівнів категорій.

(9) Імпутація пропусків

Умова завдання

Часто в даних трапляються пропуски. Виберіть 5 випадкових рядків у наборі даних і видаліть значення для LotArea. Реалізуйте щонайменше два підходи для заповнення (імпутації) цих значень за допомогою інструментів pandas або класу SimpleImputer / IterativeImputer з sklearn.impute. Запишіть відповідні формули/модель та наведіть мотивацію для вашого підходу. Зауважте, що для імпутації слід використовувати повний набір даних, а не лише набір змінних після відбору ознак. Зверніть увагу: ваш алгоритм має обробляти лише дійсні пропуски (як-от у LotFrontage або ваші штучні пропуски в LotArea), а не структурні категорії NA, які ви мали зберегти як окремі факторні рівні.

Код Python

```python
# Повний df: усі 2930 рядків і всі колонки. Ховаємо LotArea у 5 випадкових рядках.
masked_index = rng.choice(df.index.to_numpy(), size=5, replace=False)
true_lotarea = df.loc[masked_index, "LotArea"]
df_missing = df.copy()
df_missing["LotArea"] = df_missing["LotArea"].astype(float)
df_missing.loc[masked_index, "LotArea"] = np.nan
# Заповнюємо лише числові NaN. Категоріальні "NA" не чіпаємо; GarageYrBlt без гаража
# не є пропуском; MSSubClass – коди категорій; SalePrice – відгук, який пояснюємо.
impute_columns = [c for c in df.select_dtypes("number").columns
                  if c not in ["Order", "PID", "SalePrice", "MSSubClass", "GarageYrBlt"]]
# Підхід 1: медіана колонки. Підхід 2: кожну колонку з пропусками регресуємо на решту.
median_filled = df_missing.copy()
median_filled[impute_columns] =
      SimpleImputer(strategy="median").fit_transform(df_missing[impute_columns])
iterative_filled = df_missing.copy()
iterative_filled[impute_columns] = IterativeImputer(max_iter=25,
      random_state=SEED).fit_transform(df_missing[impute_columns])
# Далі: таблиці порівняння з істинними значеннями; assert, що змінено лише пропуски.
```

```
Колонок для імпутації: 34; пропуски: {'LotFrontage': 490, 'LotArea': 5, 'MasVnrArea': 23, 'BsmtFinSF1': 1, 'BsmtFinSF2': 1, 'BsmtUnfSF': 1, 'TotalBsmtSF': 1, 'BsmtFullBath': 2, 'BsmtHalfBath': 2, 'GarageCars': 1, 'GarageArea': 1}
```

Order	Neighborhood	Справжня LotArea	Медіана	IterativeImputer
2266	Timber	13758	9434	6488.77
1286	OldTown	10440	9434	8673.19
1917	NAmes	8760	9434	9665.54
262	CollgCr	10650	9434	12519.2
1269	NAmes	7200	9434	7612.11

	MAE, кв. фути	Медіана |помилки|, кв. фути
Медіана	1890.8	1216
IterativeImputer	2444.58	1766.81

```
LotFrontage: медіана = 68.0 | IterativeImputer, діапазон заповнених: {'min': 44.7, 'median': 70.6, 'max': 258.7}
```

Результати та інтерпретація

Пропуски створено в копії повного `df` (2930 рядків, усі колонки). П'ять випадкових рядків вибрано через `rng` із `SEED = 42`: Order 2266, 1286, 1917, 262, 1269. Справжні значення збережено лише для перевірки.

Що вважаємо пропуском: лише `NaN` у числових колонках. Це справжні пропуски `LotFrontage` (490), `MasVnrArea` (23), кілька пропусків у колонках підвалу й гаража та наші 5 штучних у `LotArea`. Не чіпаємо:

- категоріальні `"NA"` (немає алеї, басейну, підвалу...). Це окремі рівні, а не пропуски; `keep_default_na=False` зберіг їх як текст;
- `GarageYrBlt` будинків без гаража: такого року не існує;
- `SalePrice` (відгук, який потім пояснюємо) і `MSSubClass` (коди категорій).

Імпутація використовує 34 числові колонки.

Підхід 1, медіана (`SimpleImputer(strategy="median")`):

$$\widehat{x}_i = \text{median}\{x_j : \ x_j \text{ спостережено}\}.$$

де $\widehat{x}_i$ позначає заповнене значення, x_j спостережені значення тієї самої колонки. Для `LotArea` це 9434 кв. фути, для `LotFrontage` 68 футів. Медіана проста й не залежить від кількох дуже великих ділянок, які сильно змістили б середнє. Але всі пропуски отримують одне значення, а інші ознаки будинку ігноруються.

Підхід 2, `IterativeImputer`: кожну колонку з пропусками по черзі прогнозуємо лінійною регресією на всі інші колонки:

$$\widehat{x}_{i,LotArea} = b_0 + \sum_j b_j x_{ij}.$$

де x_{ij} позначає значення інших числових колонок рядка i, b_0 і b_j коефіцієнти регресії `LotArea` на них. Спочатку пропуски заповнюються середніми, далі значення уточнюються раунд за раундом (до 25 раундів), доки зміни не стануть малими. Модель за замовчуванням `BayesianRidge`, тобто лінійна регресія з невеликим штрафом на коефіцієнти. Мотивація: площа ділянки пов'язана з фасадом, площею будинку, гаражем тощо. Тому імпутацію робимо на повному наборі, а не лише на змінних після відбору: інші змінні містять інформацію про пропущене значення.

Результат на 5 прихованих значеннях: середня абсолютна помилка (MAE) медіани становить 1891 кв. фут, `IterativeImputer` 2445. Тут проста медіана виявилася точнішою; регресійний метод найбільше помилився для Order 2266 (13758 $\rightarrow$ 6489). П'яти значень замало для загального висновку. Можлива причина: лінійна модель на початковій шкалі погано описує скошену `LotArea`; варто було б імпутувати $\log(LotArea)$.

Обидва методи дають одне «найкраще» значення й не показують невизначеності пропуску; медіана ще й зменшує розкид змінної. Подальші регресії продовжуємо на тих самих 2414 повних спостереженнях, щоб моделі порівнювалися на однакових даних.

(10) Ціна чи логарифм ціни

Умова завдання

Тепер розглянемо модель із SalePrice та модель із log(SalePrice). Запустіть відповідний тест (наприклад, тест Девідсона-Маккіннона або порівняння через трансформовані залишки), щоб вирішити, яка з моделей є кращою. Поясніть ідею цього тесту і чому ви не можете прийняти аналогічне рішення за допомогою простого порівняння критеріїв AIC/BIC, виданих statsmodels для цих двох моделей напряму.

Код Python

```python
level_model = selected_model
log_model = sm.OLS(np.log(y), X_selected).fit()
n_obs = len(y)
# Двостороннє порівняння через штучні регресори (тест Девідсона–Маккіннона).
# Прогнози альтернативної моделі переводимо в шкалу залежної змінної.
assert (level_model.fittedvalues > 0).all()  # без цього log(прогнозу) не визначений
aux_level = sm.OLS(y, X_selected.assign(alternative=np.exp(log_model.fittedvalues) / 1000)).fit()
aux_log = sm.OLS(np.log(y),
     X_selected.assign(alternative=np.log(level_model.fittedvalues))).fit()
comparison_rows = []
for name, auxiliary in [("SalePrice ~ X", aux_level), ("log(SalePrice) ~ X", aux_log)]:
    robust = auxiliary.get_robustcov_results(cov_type="HC1", use_t=True)
    position = auxiliary.model.exog_names.index("alternative")
    comparison_rows.append({"Нульова специфікація": name,
                            "t звичайний": auxiliary.tvalues["alternative"],
                            "p звичайне": auxiliary.pvalues["alternative"],
                           "t HC1": robust.tvalues[position], "p HC1": robust.pvalues[position]})
# AIC/BIC лог-моделі переводимо до щільності самої ціни через якобіан 1/y.
jacobian = np.log(y).sum()
# Далі: збирання таблиць тесту та AIC/BIC з поправкою (виводи нижче).
```

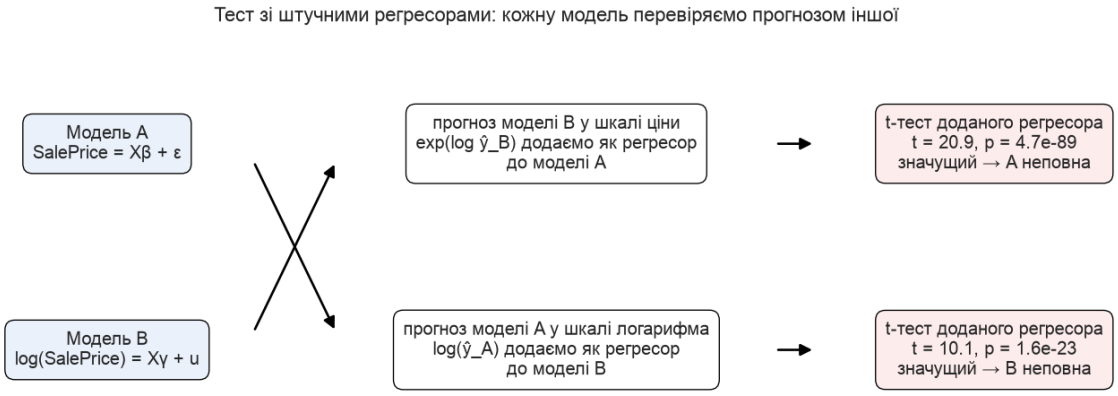


Рис. 5: Схема тесту зі штучними регресорами: прогноз кожної моделі переводимо в шкалу іншої.

Нульова специфікація	t звичайний	p звичайне	t HC1	p HC1
SalePrice ~ X	20.9046	4.68683e-89	6.42438	1.60109e-10
log(SalePrice) ~ X	10.1076	1.55684e-23	5.17715	2.44653e-07

	AIC зі statsmodels	AIC для щільності y	BIC для щільності y
SalePrice	55370.7	55370.7	55897.5
log(SalePrice)	-3214.31	54756.8	55283.6

Результати та інтерпретація

Порівнюємо дві моделі з тими самими 28 відібраними змінними:

$$(A)\ y = X\beta + \varepsilon, \qquad (B)\ \log y = X\gamma + u, \qquad y = SalePrice.$$

де X позначає матрицю відібраних регресорів із константою, β і γ вектори коефіцієнтів, ε і u похибки.

Тест зі штучними регресорами виконуємо в обох напрямках (схема на рисунку вище):

1. До моделі y додаємо прогнози лог-моделі, переведені у шкалу ціни: $\exp(\widehat{\log y})$.
2. До моделі $\log y$ додаємо логарифм прогнозів моделі ціни: $\log(\widehat{y})$. Перед цим перевіряємо, що всі $\widehat{y} > 0$.
3. В обох допоміжних регресіях перевіряємо H_0: коефіцієнт доданого регресора дорівнює нулю. Якщо він значущий, поточна специфікація не врахувала інформацію, яку дає альтернативна.

Ділення доданого регресора на 1000 у першій регресії змінює лише масштаб його коефіцієнта, а не t-статистику. Експонента прогнозу логарифма сама по собі не є оцінкою середньої ціни без додаткових припущень; тут це допоміжний регресор для перевірки специфікації.

Код показує звичайні t-тести й додатково тести зі стандартними похибками HC1, стійкими до гетероскедастичності. HC1 змінює стандартні похибки, а не оцінки коефіцієнтів; висновок залишається наближеним.

Обидві специфікації відхиляються на рівні 0.05, зокрема й з HC1. Тест не дає однозначного переможця: відхилення моделі A ще не доводить правильності B. Це один із можливих результатів такого порівняння. Дані вказують, що обидві прості форми залишають невраховані закономірності.

Чому не можна порівнювати AIC/BIC зі statsmodels напряму. Вони стосуються різних залежних змінних і різних щільностей. Для лог-моделі треба перейти від щільності $\log y$ до щільності y:

$$f_Y(y) = f_{\log Y}(\log y)/y, \qquad \ell_B^{(y)} = \ell_B - \sum_i \log y_i.$$

де f_Y і $f_{\log Y}$ позначають щільності ціни та її логарифма, ℓ_B логарифм правдоподібності моделі B для $\log y$, $\ell_B^{(y)}$ її логарифм правдоподібності у шкалі y.

Тому до AIC та BIC лог-моделі додаємо $2\sum_i \log y_i$.

Після поправки обидва критерії віддають перевагу лог-моделі серед цих двох кандидатів. Але це не скасовує результат тестів: жодну з них не можна вважати підтвердженою правильною специфікацією. Менший AIC також не гарантує меншої MSE прогнозу ціни в доларах.

RSS мають різні одиниці, а R^2 безрозмірний, проте описує варіацію різних відгуків. Тому їхнє пряме порівняння для y і $\log y$ некоректне. Через важкі хвости, впливові точки та попередній відбір змінних наведені p-значення слід сприймати як наближену діагностику.

(11) Крос-валідація

Умова завдання

Ми хочемо оцінити прогнозну здатність побудованої регресії. Розглянемо повну модель та модель після відбору ознак. Порівняйте ці дві моделі за допомогою крос-валідації «залишення по одному» (LeaveOneOut) та 5-блочної крос-валідації (KFold) з бібліотеки sklearn.model_selection. Поясніть ідею цієї техніки за допомогою формул та зробіть висновок про прогностичну здатність обох моделей на основі метрики MSE.

Код Python

```python
X_full_cv = X_full.drop(columns="const")        # LinearRegression сама додає константу
X_selected_cv = X_selected.drop(columns="const")
splitters = {"LeaveOneOut": LeaveOneOut(),
             "KFold (5 блоків)": KFold(n_splits=5, shuffle=True, random_state=SEED)}
models = {"Повна": (X_full_cv, full_model), "Після відбору": (X_selected_cv, selected_model)}
cv_rows, cv_errors = [], {}
# cross_val_predict навчає модель без тестового блоку і прогнозує саме його.
for scheme, splitter in splitters.items():
    for name, (X_cv, fitted_model) in models.items():
        prediction = cross_val_predict(LinearRegression(), X_cv, y, cv=splitter, n_jobs=-1)
        cv_errors[(scheme, name)] = y - prediction
        mse = np.mean((y - prediction) ** 2)
        cv_rows.append({"Схема": scheme, "Модель": name, "Параметрів": X_cv.shape[1] + 1,
                        "MSE навчальне, дол.²": fitted_model.ssr / n_obs,
                        "MSE крос-валідації, дол.²": mse, "RMSE, дол.": np.sqrt(mse)})
cv_results = pd.DataFrame(cv_rows)
# Далі: assert збіги LinearRegression і OLS, найбільші помилки LOO, графік.
```

Схема	Модель	Параметрів	MSE навчальне, дол.²	MSE крос-валідації, дол.²	RMSE, дол.
LeaveOneOut	Повна	269	4.18001e+08	9.55734e+08	30915
LeaveOneOut	Після відбору	91	4.96967e+08	9.15862e+08	30263.2
KFold (5 блоків)	Повна	269	4.18001e+08	9.55302e+08	30908
KFold (5 блоків)	Після відбору	91	4.96967e+08	9.13883e+08	30230.5

Повна	Після відбору	Order
-660415	-650715	1499
-549931	-540751	2181
-439550	-431852	2182
199807	195993	514
154980	168660	1761

```
MSE LOO без трьох найбільших помилок (Order [1499, 2181, 2182] ): {'Повна': '5.705e+08', 'Після відбору': '5.427e+08'}
```

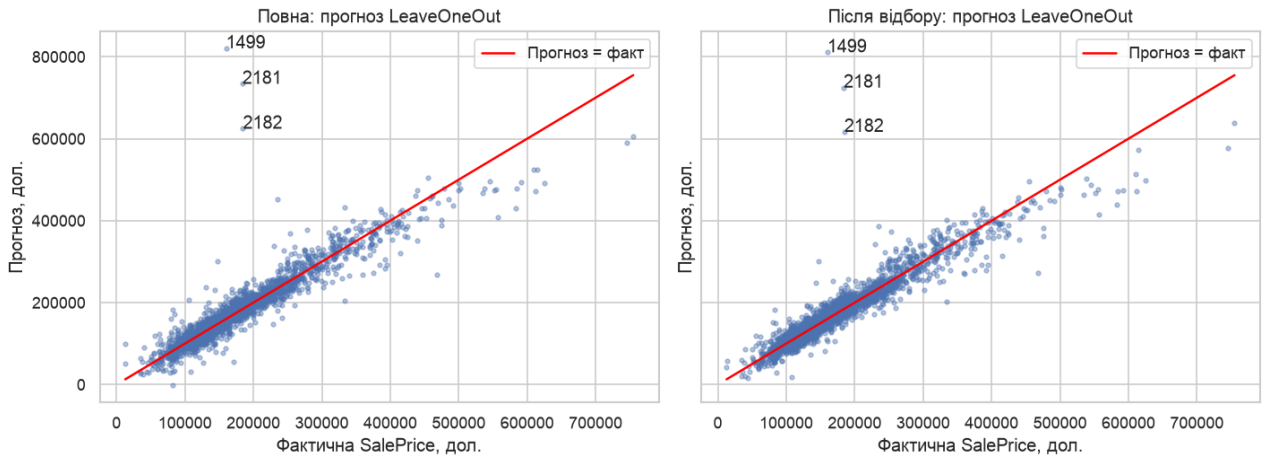


Рис. 6: Фактичні ціни та прогнози LeaveOneOut для двох моделей.

Результати та інтерпретація

Ідея крос-валідації: кожен прогноз отримуємо з моделі, яку навчали без відповідного тестового рядка. Навчальна MSE зазвичай оптимістична, бо коефіцієнти вже підігнано до цих цін.

LeaveOneOut: для кожного i навчаємо модель на $n-1$ рядках і прогнозуємо пропущений:

$$MSE_{LOO} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i^{(-i)})^2.$$

де $\widehat{y}_i^{(-i)}$ позначає прогноз для рядка i з моделі, навчаної без нього, $n = 2414$. Це 2414 підгонок для кожної моделі.

KFold: ділимо рядки на 5 блоків $V_1, \ldots, V_5$ (`shuffle=True`, `SEED=42`). Кожен блок один раз є тестовим, інші чотири навчальними:

$$MSE_5 = \frac{1}{n}\sum_{v=1}^{5}\sum_{i\in V_v}(y_i - \widehat{y}_i^{(-V_v)})^2.$$

де $\widehat{y}_i^{(-V_v)}$ позначає прогноз із моделі, навчаної без блоку V_v, до якого належить рядок i. Це 5 підгонок для кожної моделі. Обидві моделі використовують ті самі розбиття.

Порівнюємо повну та відібрану моделі ціни в доларах, як вимагає пункт 11. У пункті 10 лог-модель мала кращі інформаційні критерії, але обидві специфікації були відхилені; її прогнозну MSE тут не оцінюємо. `LinearRegression` і OLS дають однакові підгонки для тієї самої матриці регресорів.

Висновок:

- На навчальних даних повна модель має меншу MSE. Для вкладених моделей на тих самих рядках додавання регресорів не може збільшити мінімальний RSS.
- У цих розбиттях відібрана модель має меншу MSE: приблизно на 4.2% для LOO і на 4.3% для KFold. Вона простіша й тут прогнозує трохи точніше; це узгоджується з ризиком перенавчання повної моделі, але не доводить, що всі додаткові параметри підганяли лише шум.
- LOO та KFold дали близькі оцінки. Для відібраної моделі RMSE становить приблизно 30.3 тис. дол.; це корінь із середнього квадрата помилки, а не гарантія такої похибки для кожного будинку.
- Великий внесок у MSE дають Order 1499, 2181 і 2182: помилки відібраної моделі приблизно 651, 541 і 432 тис. дол. Без їхніх помилок MSE LOO була б $5.705 \cdot 10^8$ та $5.427 \cdot 10^8$ для повної й відібраної моделей. Це лише аналіз внеску точок у метрику; у головному результаті всі продажі залишено.
- Якщо індикатор категорії дорівнює 1 лише для тестового будинку, у навчальній частині весь його стовпець нульовий. `LinearRegression` задає такому коефіцієнту 0: модель не може навчитися ефекту відсутнього рівня. Це пояснює великі помилки для Order 1499 і 2181.

Обмеження: набір змінних обрано за BIC на всіх 2414 рядках. Тому це порівняння двох зафіксованих специфікацій, а не незалежна оцінка всього алгоритму відбору. Для останньої відбір треба повторювати всередині кожної навчальної частини; поточна оцінка відібраної моделі може бути оптимістичною. Різниця близько 4% сама по собі не доводить статистичної переваги на інших вибірках.

Задача 6: Нелінійні регресії в Python

За умовою використовуємо лише перші 500 продажів та пояснюємо SalePrice через YearBuilt. У цих двох колонках немає пропусків. Файл упорядкований за роком продажу, тому ця підвибірка містить лише продажі 2009–2010 років і не є випадковою.

(a) Лінійна модель

Умова завдання

Побудуйте двовимірну діаграму розсіювання (bivariate scatter plot) та оцініть відповідну лінійну (!) модель через statsmodels або LinearRegression з sklearn. Додайте лінію регресії на графік.

Код Python

```python
from scipy.optimize import curve_fit
# За умовою беремо перші 500 рядків файлу.
df_500 = df.iloc[:500].copy()
x = df_500["YearBuilt"].to_numpy(dtype=float)
y = df_500["SalePrice"].to_numpy(dtype=float)
n = len(y)
# МНК через формулу statsmodels; fittedvalues — прогнози на тих самих 500 продажах.
linear_model = smf.ols("SalePrice ~ YearBuilt", data=df_500).fit()
linear_pred = linear_model.fittedvalues.to_numpy()
years = np.linspace(x.min(), x.max(), 300)
fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(x, y, s=10, alpha=0.4, label="Продажі (перші 500)")
ax.plot(years, linear_model.params["Intercept"] + linear_model.params["YearBuilt"] * years,
        color="red", label="Лінійна регресія")
plt.show()
# Далі: describe(), роки продажу, таблиці оцінок і залишків, оформлення графіка.
```

```
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept  -3.46e+06   2.28e+05    -15.149      0.000   -3.91e+06   -3.01e+06
YearBuilt  1846.6549    115.494     15.989      0.000    1619.740    2073.570
==============================================================================
```

	Лінійна модель
beta_0	-3.45966e+06
beta_1 (YearBuilt)	1846.65
SE(beta_1)	115.494
t	15.9893
p	9.55339e-47
R^2	0.339221
Найстаріший рік	1875
Його фактична ціна	94000
Його прогноз	2818.23

YearBuilt	Продажів	Середній залишок, дол.	Частка точок під прямою
(1870, 1950]	79	22519.3	0.329114
(1950, 1980]	185	-25041.8	0.8
(1980, 2000]	93	-4279.42	0.645161
(2000, 2010]	143	22739.1	0.524476

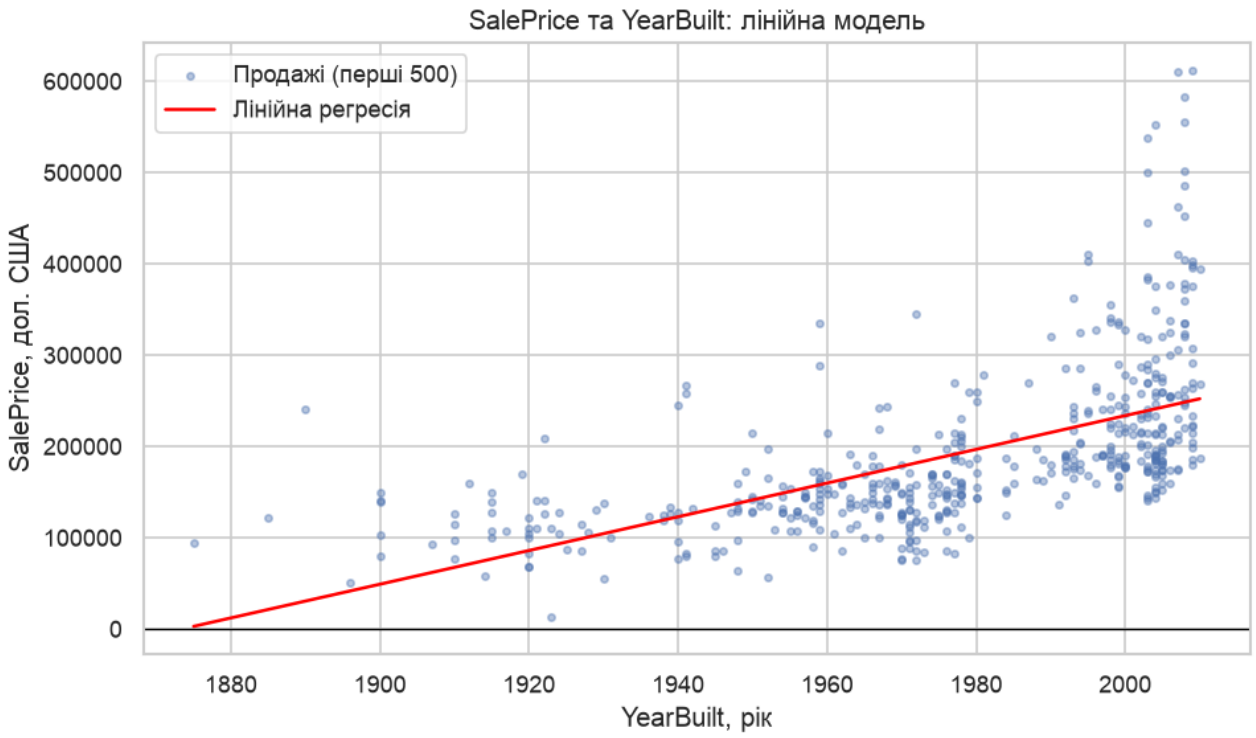


Рис. 7: Лінійна модель для перших 500 продажів.

Результати та інтерпретація

Модель: $SalePrice_i = \beta_0 + \beta_1 YearBuilt_i + \varepsilon_i$, де β_0 позначає вільний член, β_1 нахил, ε_i похибку продажу i; оцінена МНК через `statsmodels`.

- $\widehat{\beta}_1 = 1846.65$ ($SE = 115.5$, $t = 15.99$, $p \approx 10^{-46}$): будинок, збудований на рік пізніше, у середньому дорожчий на 1847 дол.
- $\widehat{\beta}_0 = -3.46$ млн, тобто «ціна» будинку року 0. Економічного змісту вона не має: це продовження прямої далеко за межі даних.
- $R^2 = 0.339$: рік будівництва пояснює третину варіації цін.

Що видно на графіку. Хмара точок опукла: до 1950 р. ціни ростуть повільно, після 1990 р. швидко. Пряма цього не враховує, тому залишки мають систематичний візерунок:

- до 1950 р. пряма занижує ціни (середній залишок +22.5 тис. дол.);
- у 1951–1980 рр. 80% точок лежать під прямою (середній залишок -25 тис.);
- після 2000 р. пряма знову занижує ціни (+22.7 тис.).

Для найстарішого будинку (1875 р.) пряма прогнозує лише 2818 дол., хоча його продано за 94000; продовжена вліво, вона дала б від'ємні ціни. Крім того, розкид цін для нових будинків значно більший. Усе це вказує, що лінійна форма тут не найкраща.

(б) Експоненційна модель

Умова завдання

Тепер оцініть відповідну нелінійну регресію (наприклад, експоненціальну або степеневу залежність) за допомогою функції curve_fit з бібліотеки scipy.optimize. Додайте криву регресії на графік і порівняйте (кількісно) якість підгонки нелінійної моделі з лінійною за допомогою метрик R^2 та MSE.

Код Python

```python
YEAR_0 = 1970  # опорний рік: a — прогноз ціни будинку, збудованого в 1970 р.
def exponential(year, a, b):
    """SalePrice = a * exp(b * (YearBuilt - 1970))."""
    return a * np.exp(b * (year - YEAR_0))
# Початкові значення: якщо log(SalePrice) = log(a) + b * (YearBuilt - 1970),
# то їх дає звичайна лінійна регресія логарифма ціни.
log_fit = sm.OLS(np.log(y), sm.add_constant(x - YEAR_0)).fit()
p0 = [np.exp(log_fit.params[0]), log_fit.params[1]]
params, pcov = curve_fit(exponential, x, y, p0=p0)
a_hat, b_hat = params
exp_pred = exponential(x, a_hat, b_hat)
# R² і MSE рахуємо однаково для обох моделей на тих самих y.
def r_squared(prediction):
    return 1 - np.sum((y - prediction) ** 2) / np.sum((y - y.mean()) ** 2)
def mse(prediction):
    return np.mean((y - prediction) ** 2)
# Далі: assert R² лінійної моделі, таблиці порівняння, контрольні роки, графік.
```

	Початкове значення p0	Оцінка curve_fit
a	161996	163627
b	0.0101651	0.0129343

```
Річне зростання прогнозу: 100*(exp(b) − 1) = 1.302%
```

	Параметрів	R^2	MSE, дол.^{2}	RMSE, дол.
Лінійна	2	0.339221	5.27658e+09	72640.1
Експоненційна	2	0.392684	4.84965e+09	69639.5

YearBuilt	Лінійна, дол.	Експоненційна, дол.
1880	12051.5	51085.3
1920	85917.7	85701.6
1960	159784	143774
2000	233650	241198
2010	252117	274503

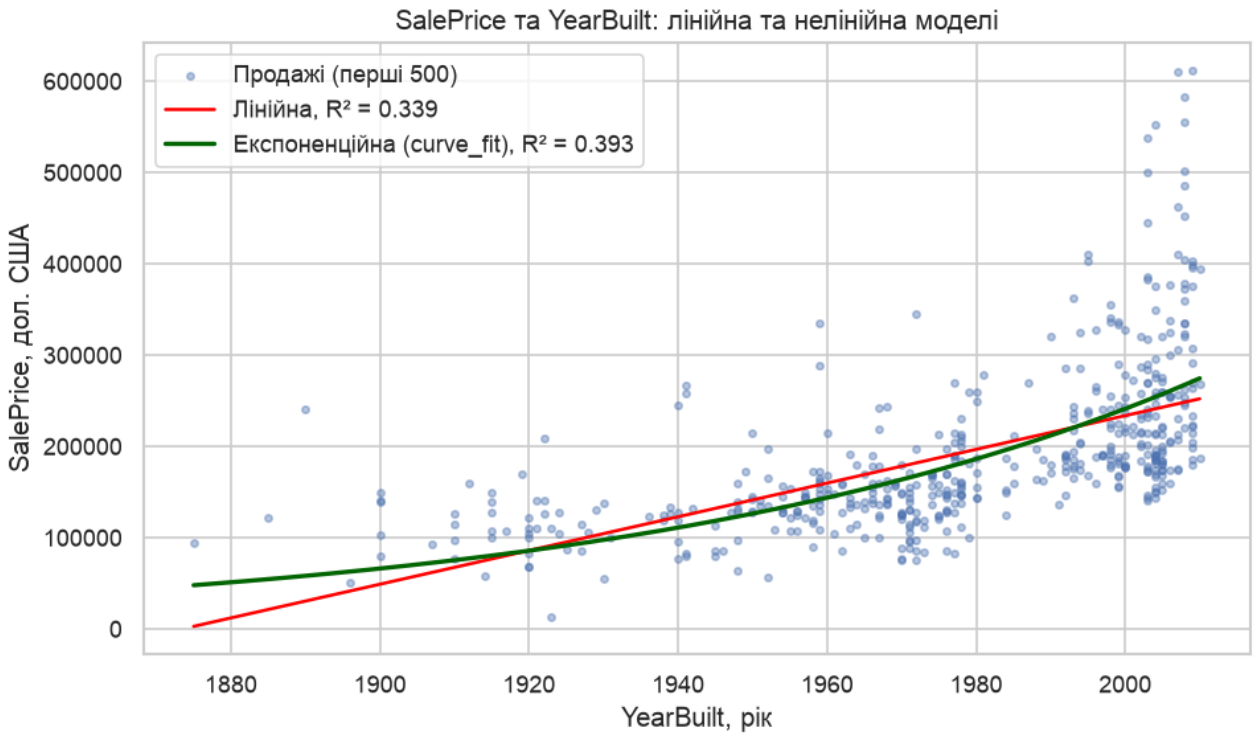


Рис. 8: Порівняння лінійної та експоненційної моделей.

Результати та інтерпретація

Модель:

$$SalePrice_i = a\, e^{\,b\,(YearBuilt_i - 1970)} + \varepsilon_i.$$

де a позначає ціну будинку 1970 року за моделлю, b темп зростання за рік, ε_i похибку. Чому експонента: вона описує зростання ціни на сталий відсоток за кожен пізніший рік будівництва, має опуклу форму, як хмара точок, і за $a > 0$ дає додатні прогнозовані ціни. Рік зсунуто на 1970 (близько до середнього 1977): тоді a є прогнозом ціни будинку 1970 р., а обчислення стабільніші.

`curve_fit` мінімізує

$$RSS(a, b) = \sum_i \left(y_i - a\, e^{\,b(x_i - 1970)}\right)^2$$

ітераційно (за замовчуванням методом Левенберга–Марквардта), починаючи з початкових значень `p0`; тут x_i позначає рік будівництва, а y_i ціну продажу i. Явної формули, як у лінійній регресії, немає. `p0` взято з лінійної регресії $\log(SalePrice)$ на $(YearBuilt - 1970)$: $a_0 = 161996$, $b_0 = 0.0102$.

Оцінки: $\widehat{a} = 163627$ дол. (прогноз для будинку 1970 р.), $\widehat{b} = 0.01293$. Кожен пізніший рік будівництва збільшує прогноз на $100(e^{\widehat{b}} - 1) = 1.30\%$. Оцінка $\widehat{b}$ більша за початкову 0.0102. NLS мінімізує квадрати помилок у доларах, тож дорогі нові будинки з великими відхиленнями важать більше; регресія логарифма мінімізує квадрати логарифмічних помилок, які для малих відхилень наближено відповідають відносним помилкам.

Порівняння на тих самих 500 продажах (обидві моделі мають по 2 параметри, тож скоригований R^2 не потрібен):

$$R^2 = 1 - \frac{\sum_i (y_i - \widehat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \qquad MSE = \frac{1}{n}\sum_i (y_i - \widehat{y}_i)^2.$$

де $\widehat{y}_i$ позначає прогноз відповідної моделі, $\bar{y}$ середню ціну, $n = 500$.

Експоненційна модель краща: MSE менша на 8.1%, R^2 вищий на 0.053. Крива краще передає повільне зростання цін старих будинків і швидке зростання цін нових. Для будинку 1880 р. вона прогнозує 51 тис. дол. замість 12 тис. у прямої, для 2010 р. 275 тис. замість 252 тис.

Проте покращення помірне: обидві моделі пояснюють менше 40% варіації, бо ціну визначають також площа, якість, район тощо. Для нелінійної моделі R^2 є лише описовою мірою: розклад $TSS = ESS + RSS$ тут не виконується, і R^2 навіть може бути від'ємним. Порівнювати R^2 двох моделей тут коректно, бо вони мають ті самі y і n; це рівнозначно порівнянню MSE.

(в) Статистичні висновки для NLS

Умова завдання

Поясніть своїми словами, чому всі класичні тести та статистичні висновки (наприклад, точні t-розподіли для малих вибірок) не є прямо застосовними до оцінок нелінійного методу найменших квадратів (NLS), отриманих через curve_fit.

Код Python

```python
# curve_fit оцінює коваріацію через лінеаризацію: pcov = s^2 (J^T J)^(-1),
# де J — похідні моделі за a і b в точці оцінки, s^2 = RSS / (n − 2).
t = x - YEAR_0
J = np.column_stack([np.exp(b_hat * t), a_hat * t * np.exp(b_hat * t)])
s2 = np.sum((y - exp_pred) ** 2) / (n - 2)
se = np.sqrt(np.diag(pcov))
z = stats.norm.ppf(1 - ALPHA / 2)
# Нормальність залишків NLS перевіряємо тим самим тестом Jarque–Bera.
residuals = y - exp_pred
jb = stats.jarque_bera(residuals)
# Далі: assert pcov = s²(JᵀJ)⁻¹, таблиця оцінок, розкид залишків за періодами, графік.
```

	Оцінка	SE з pcov (наближена)	Оцінка / SE	Наближений 95% ДІ: від	до
a	163627	3854.67	42.449	156072	171182
b	0.0129343	0.000813133	15.9068	0.0113406	0.0145281

```
Кореляція оцінок a і b: −0.748
Jarque–Bera для залишків NLS: JB = 672.3, p = 1.04e−146
```

	Стандартне відхилення залишків, дол.
(1870, 1950]	46474.6
(1950, 1980]	43587
(1980, 2000]	60448
(2000, 2010]	102347

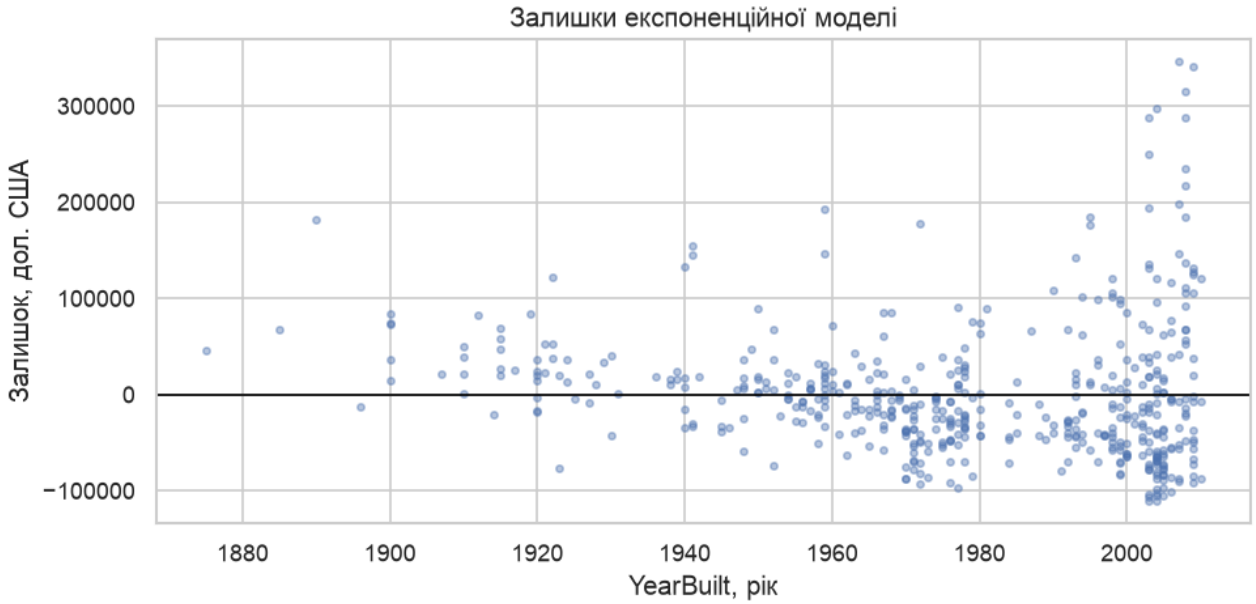


Рис. 9: Залишки експоненційної моделі за роком будівництва.

Результати та інтерпретація

Чому в лінійній регресії висновки точні. За фіксованих X оцінка

$$\widehat{\beta} = (X^T X)^{-1} X^T y$$

є лінійною функцією y, де X позначає матрицю регресорів, а y вектор відгуку. Якщо похибки нормальні, $\varepsilon \sim N(0, \sigma^2 I)$, то $\widehat{\beta}$ точно нормальна, а $(n-k)s^2/\sigma^2 \sim \chi^2(n-k)$ і не залежить від $\widehat{\beta}$. Тому $t = (\widehat{\beta}_j - \beta_j)/SE(\widehat{\beta}_j)$ має точний t-розподіл для будь-якого n, навіть $n = 10$. Так само точні й F-тести.

Чому для NLS це не так. Оцінка $\widehat{\theta} = (\widehat{a}, \widehat{b})$ не має формули: її знаходить ітераційний алгоритм, і вона є нелінійною функцією y. Навіть із нормальними похибками $\widehat{\theta}$ не є точно нормальною, у малих вибірках може бути зміщеною, а s^2 не має точного розподілу χ^2. Тому точних t- і F-розподілів немає.

Що дає `curve_fit`. Матриця `pcov` обчислюється через лінеаризацію: біля $\widehat{\theta}$ модель замінюють лінійною з «регресорами»-похідними $J = \left[\partial f/\partial a,\ \partial f/\partial b\right] = \left[e^{bt},\ a\,t\,e^{bt}\right]$:

$$\widehat{Cov}(\widehat{\theta}) \approx s^2 (J^T J)^{-1}, \qquad s^2 = \frac{RSS}{n-2}.$$

де J позначає матрицю похідних, $t = YearBuilt - 1970$, s^2 оцінку дисперсії похибок, RSS суму квадратів залишків NLS. Це асимптотичне наближення: воно добре, коли n велике, а модель майже лінійна в межах невизначеності параметрів. Звідси $SE(\widehat{a}) = 3855$, $SE(\widehat{b}) = 0.00081$ і наближений 95% інтервал для b: $[0.0113;\ 0.0145]$, що після перетворення $100(e^b - 1)$ дає приблизно 1.14–1.46% за кожен пізніший рік будівництва. Тут ми свідомо беремо нормальний квантиль 1.96, а не точний t-квантиль. Самих p-значень `curve_fit` не дає.

Наші дані додатково порушують припущення:

- залишки ненормальні: Jarque–Bera $= 672$, $p \approx 10^{-146}$;
- розкид залишків зростає з роком: стандартне відхилення близько 44–46 тис. дол. до 1980 р., 60 тис. у 1981–2000 рр. і 102 тис. після 2000 р. (графік). Формула `pcov` припускає однакову дисперсію;

- перші 500 рядків не є випадковою вибіркою.

Тому навіть наближені стандартні похибки й інтервали слід сприймати обережно.

(г) Збіжність і перенавчання

Умова завдання

Які проблеми з обчислювальною збіжністю (convergence) та перенавчанням можуть виникнути, якщо ми вирішимо побудувати складну нелінійну регресію, використовуючи одночасно всі наявні пояснювальні змінні у датасеті?

Код Python

```python
import warnings
# 1) Збіжність: та сама крива, але без зсуву року.
def exponential_raw(year, a, b):
    return a * np.exp(b * year)
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    raw_default, raw_default_cov = curve_fit(exponential_raw, x, y)  # p0 = [1, 1] за замовчуванням
    with np.errstate(over="ignore"):
        raw_default_mse = mse(exponential_raw(x, *raw_default))
# Та сама крива без зсуву року, але з p0, перерахованим із центрованої моделі.
raw_params, raw_cov = curve_fit(exponential_raw, x, y, p0=[p0[0] * np.exp(-p0[1] * YEAR_0),
    p0[1]])
raw_se = np.sqrt(np.diag(raw_cov))
# 2) Усі пояснювальні змінні перших 500 рядків: кількість параметрів проти n.
features = df_500.drop(columns=["Order", "PID", "SalePrice"])
categorical = ["MSSubClass"] + list(features.select_dtypes(exclude="number").columns)
X_all = pd.get_dummies(features, columns=categorical, drop_first=True, dtype=float)
complete = X_all.notna().all(axis=1)
X_complete = X_all[complete]
# Далі: друк попереджень, таблиці порівняння параметризацій і розмірів матриці.
```

```
Без p0 curve_fit повернув: [1. 1.] | MSE: inf
Попередження: ['OptimizeWarning: Covariance of the parameters could not be estimated', 'RuntimeWarning: overflow
encountered in exp']
```

	a	SE(a) / a	b	Кореляція a і b	MSE
a*exp(b*(year - 1970))	163627	0.0235577	0.0129343	-0.74794	4.84965e+09
a*exp(b*year), добре p0	1.40509e-06	1.6199	0.0129344	-0.999953	4.84965e+09

	Значення
Рядків	500
Стовпців після кодування категорій	234
Повних рядків (без NaN)	393
Стовпців, що в повних рядках завжди 0	16
Стовпців, ненульових лише в 1 рядку	19
Ранг [1, X] на повних рядках	204

Результати та інтерпретація

Проблеми зі збіжністю.

- NLS не має явного розв'язку: алгоритм рухається від `p0` до мінімуму $RSS(\theta)$. Ця функція може мати кілька локальних мінімумів і пласкі «долини». Алгоритм може зупинитися не там, розійтися або сильно залежати від `p0`. З усіма змінними параметрів сотні, і кожен крок потребує матриці похідних J розміру $n \times p$. Підібрати добрі початкові значення для багатьох параметрів складніше; варто перевіряти кілька початкових наближень.

- Навіть одна змінна показує проблему. Ту саму експоненту без зсуву року, $a\,e^{b\cdot YearBuilt}$, запустили зі стандартним `p0 = [1, 1]`. Тоді $e^{1\cdot 2000}$ переповнюється, і `curve_fit` повернув $[1, 1]$ без змін, з MSE $= \infty$, лише з попередженнями, без помилки. З добрим `p0` ця версія знаходить ту саму криву (MSE однакова, $4.85 \cdot 10^9$). Але $\widehat{a} = 1.4 \cdot 10^{-6}$, відносна похибка $\widehat{a}$ сягає 162%, а кореляція $\widehat{a}$ і $\widehat{b}$ дорівнює -0.99995: такий запис чисельно незручний, а прогноз для року 0 є далекою екстраполяцією. Зі зсувом на 1970 кореляція -0.75, а відносна похибка 2.4%. Змінні Ames мають дуже різні масштаби (площі ~10^4, роки ~2000, індикатори 0/1), тож таких проблем стане значно більше. Потрібні центрування й масштабування, обмеження на параметри, розумні `p0`.
- Не всі параметри можна оцінити. Для перших 500 рядків кодування всіх змінних дає 234 стовпці. `curve_fit` не приймає `NaN`, а повних рядків лише 393. У цих рядках 16 індикаторів завжди дорівнюють 0, а 19 ненульові лише в одному рядку. Ранг матриці $[1, X]$ дорівнює 204 замість 235, тобто навіть лінійна модель не має єдиного розв'язку. Для конкретної нелінійної моделі треба окремо перевіряти ранг її якобіана J: з рангу X він автоматично не випливає. Якщо стовпці J залежні, деякі параметри неможливо оцінити окремо; `curve_fit` може видати попередження про коваріацію або дуже великі стандартні похибки.

Перенавчання.

- Потенційних параметрів багато відносно кількості спостережень (235 стовпців на 393 повні рядки), а складні нелінійні форми можуть додати гнучкості. Зростає ризик підгонки шуму: навчальний R^2 може бути високим, а помилка на нових будинках великою. Це ризик, а не гарантований результат; його перевіряють на відкладених даних або крос-валідацією.
- У задачі 5 рідкісні категорії дали точну підгонку окремих будинків ($h = 1$), а помилка крос-валідації була більшою за навчальну. Тут спостережень значно менше, тож діагностика рідкісних рівнів і перевірка прогнозів особливо важливі.

Що робити: обирати небагато змінних (відбір, як у задачі 5) і прості форми, підказані графіками; часто достатньо лінійної моделі для $\log(SalePrice)$. Також допомагає регуляризація, а якість варто перевіряти крос-валідацією, а не навчальним R^2.